

情報数理入門演習

第 4 回：1 次関数と 2 次関数 (回帰分析と最適化)

2026 年 4 月 30 日 (木)

本演習では、数学で学ぶ「1 次関数」と「2 次関数」が、データサイエンスや数理モデリングにおいてどのような役割を果たしているかを学ぶ。数式を単なる計算対象としてではなく、コンピュータを用いて現実の現象を説明し、未来を予測するための「モデル」として使いこなす視点を養う。

1 一番「自然な」予測線はどれか？

前回、複数のデータを座標平面上の「点」としてプロットする散布図について学んだ。では、その点の集まりに対して将来を予測するモデルとして、1 次関数の直線 $y = ax + b$ を当てはめたい場合、どのような直線を引くのが一番「自然」だろうか。

以下の例題のように、傾きや切片が異なる複数の直線を引いたとき、人間は視覚的にこの直線が一番データの傾向を捉えていると直感的に判断できる。単回帰分析と呼ばれる分析方法では、この人間の直感を数式化し、コンピュータを用いた計算を通して一番自然な直線を見つけさせる。

例題 1：視覚的な直線の当てはめ（加須市の気温上昇）

埼玉県有加須市を例に、ある夏の日の午前中からお昼にかけての気温の上昇傾向を見てみよう。時刻 x （時）と気温 y （°C）のデータをプロットし、そこに 3 パターンの予測線を重ねて描画する。どの直線が一番「自然（データにフィットしている）」か確認せよ。

【Python による実装と可視化】

```
import matplotlib.pyplot as plt
import numpy as np

# 実際のデータ（点）：加須市の時刻と気温
x_data = [9, 10, 11, 12, 13]
y_data = [31.5, 33.0, 35.5, 37.0, 39.5]
plt.scatter(x_data, y_data, color="red", label="Kazo Temp", zorder=5)

# 3つの予測モデル（直線）を用意
x_line = np.linspace(8, 14, 100)
y_A = 1.0 * x_line + 24.0 # 直線A
y_B = 3.0 * x_line + 2.0  # 直線B
y_C = 2.0 * x_line + 13.0 # 直線C

plt.plot(x_line, y_A, label="Line A", linestyle="--")
plt.plot(x_line, y_B, label="Line B", linestyle="--")
plt.plot(x_line, y_C, label="Line C", color="blue")

plt.xlabel("Time (Hour)")
plt.ylabel("Temperature (C)")
plt.legend()
plt.grid(True)
plt.show()
```

2 直感から数式へ

前節で人間は目で見て「直線 C が一番良い」と判断できた。では、これをコンピュータにどうやって計算させるのだろうか？ここで重要になるのが「誤差」の概念である。実際のデータ（赤い点）と、予測モデル（青い直線）との間にある「縦方向のズレ」を誤差と呼ぶ。

試しに、例題 1 の加須市のデータに対し、「 $x = 11$ のとき $y = 35.5$ を通る」という条件で直線を固定し、傾き w だけを変化させる予測モデル $y = w(x - 11) + 35.5$ を考えてみよう。

各時刻のデータにおいて「予測値と実際の値のズレ（誤差）」を計算し、プラスマイナスが打ち消し合わないようにならば 2 乗してすべて足し合わせると、以下ようになる。

$$\begin{aligned} E &= (9 \text{ 時の誤差})^2 + (10 \text{ 時の誤差})^2 + \cdots + (13 \text{ 時の誤差})^2 \\ &= (-2w + 4)^2 + (-w + 2.5)^2 + 0^2 + (w - 1.5)^2 + (2w - 4)^2 \\ &= 10w^2 - 40w + 40.5 \end{aligned}$$

このように、ズレの 2 乗和（誤差関数）は下に凸の 2 次関数（U 字型）になる性質を持つ。つまり、コンピュータにとっての一番自然な直線を引くという目的は、この 2 次関数の最小値を探す計算へと変換される。これを最適化と呼ぶ。

例題 2：実際のデータに基づく誤差関数の最小化

加須市の気温データから導かれた誤差関数 $E = 10w^2 - 40w + 40.5$ について、誤差が最小になる傾き w の値を求めよ。

【手計算（高校までのやり方）】

2 次関数の最小値を求めるため、式を変形（平方完成）する。

$$E = 10(w^2 - 4w) + 40.5 = 10(w - 2)^2 - 40 + 40.5 = 10(w - 2)^2 + 0.5$$

$(w - 2)^2 \geq 0$ であるため、 E が最小になるのは $w - 2 = 0$ のときである。

よって、最もデータにフィットする最適な傾きは $w = 2$ である。（このとき、誤差の合計は 0.5 まで小さくなる）

【Python による実装と可視化】

Python を用いれば、手計算で数式を展開しなくても、データ配列から直接「誤差の 2 乗和」を計算してグラフ化することができる。また、頂点の公式 $w = -b/(2a)$ を用いて正確な谷底を計算させる。

```
import matplotlib.pyplot as plt
import numpy as np

# 1. 加須市のデータ
x_data = np.array([9, 10, 11, 12, 13])
y_data = np.array([31.5, 33.0, 35.5, 37.0, 39.5])

# 2. 様々な傾き w を試して、それぞれ誤差 E を計算する
w_vals = np.linspace(0, 4, 100)
E_vals = []

for w in w_vals:
    # 予測モデル: y = w(x - 11) + 35.5
    y_pred = w * (x_data - 11) + 35.5
```

```
# 誤差の2乗をすべて足し合わせる (Sum of Squared Errors)
error = np.sum((y_pred - y_data)**2)
E_vals.append(error)

# 3. 頂点の公式を用いた解析的な計算 ( $10w^2 - 40w + 40.5$ )
a = 10
b = -40
w_opt = -b / (2 * a)
print("最も誤差が小さくなる傾き w:", w_opt)

# 4. 誤差関数のプロット
plt.figure(figsize=(6, 4))
plt.plot(w_vals, E_vals, color="green")
plt.axvline(w_opt, color="red", linestyle="--", label=f"Optimal w = {w_opt}")
plt.title("Error Function (Kazo City Temp)")
plt.xlabel("Slope w")
plt.ylabel("Error E")
plt.legend()
plt.grid(True)
plt.show()
```

3 1 次関数と 2 次関数を用いた交通渋滞の解析

1 次関数や 2 次関数を用いて、実世界の現象を説明できる別の例として、交通渋滞を紹介する。

道路を走る車の「密度 k (1 km あたりの台数)」と「速度 v 」の間には、道が混むほど速度が下がるという関係があるが、これを 1 次関数で表せるものであると仮定する。一方、道路の本来の目的である「流量 q (1 時間あたりに通過する車の台数)」は、「密度 $\times$ 速度」で定義される。

$$q = k \times v$$

この流量 q は、密度 k の **2 次関数 (放物線)** となり、交通工学において「基本図」と呼ばれる。この放物線の「頂点」がその道路の**最大交通容量**を示し、これを超えると流量が低下する「渋滞状態」に転移する。

例題 3：交通流の解析

ある道路において、密度 k と速度 v の関係が $v = 80 - 0.8k$ で表されたとする。このとき、流量 q を k の式で表し、さらに密度が $k = 30$ のときの流量 q を計算せよ。

【手計算 (高校までのやり方)】

流量 q は密度と速度の積であるため、関係式を代入して展開する。

$$q = k(80 - 0.8k) = -0.8k^2 + 80k$$

$k = 30$ を代入すると、

$$q = -0.8 \times 30^2 + 80 \times 30 = -720 + 2400 = 1680$$

よって、流量は **1680 台/時**である。

【Python による実装と可視化】

数式の展開および具体的な計算に加え、2 つのグラフを並べて描画し視覚的に確認する。

```
from sympy import symbols, expand
import matplotlib.pyplot as plt
import numpy as np

# 1. 数式の展開と代入計算
k_sym = symbols('k')
q_expr = k_sym * (80 - 0.8 * k_sym)
print("流量の式:", expand(q_expr))
k_val = 30
q_val = -0.8 * k_val**2 + 80 * k_val
print("k=30のときの流量:", q_val)

# 2. グラフを2枚並べてプロット
k = np.linspace(0, 100, 100)
v = 80 - 0.8 * k
q = -0.8 * k**2 + 80 * k

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(k, v, color="blue")
plt.title("Density vs Speed")
plt.grid(True)
```

```
plt.subplot(1, 2, 2)
plt.plot(k, q, color="orange")
plt.title("Density vs Flow")
plt.grid(True)

plt.tight_layout()
plt.show()
```

以下の問いについて、Google Colab を用いてプログラムを作成し、解答せよ。グラフの描画には `matplotlib.pyplot` と `numpy` を用いること。

問 1. 【前回の復習と直線の当てはめ】

ある店舗における、別商品の「気温 x 」と「売上 y 」のデータがある (x : [12, 18, 22, 28, 32], y : [80, 110, 130, 160, 180])。これを散布図としてプロットし、さらに自分が「一番自然だ」と思う予測モデル (例: $y = 4.5x + 30$ など、傾きと切片の数字を自分で決める) の直線を同じグラフ上に重ねて描画せよ。

問 2. 【1 次関数の応用：損益分岐点の計算】

ある製品を製造・販売する。製造コストの式が $y = 200x + 15000$ 、売上の式が $y = 500x$ である (x は個数)。この 2 つの 1 次関数のグラフを描画し、さらに SymPy の `solve()` を用いて、利益が出る境界線 (売上とコストが等しくなる損益分岐点) となる個数 x を計算せよ。

問 3. 【コンピュータによる探索と手計算の比較】

あるシステムのコスト関数が $C = 2x^2 - 14x + 30$ であるとする。この関数の最小値を探すため、以下の 2 つの方法を Python で実装し、結果を比較せよ。

- (1) `for` ループを用い、 x に 1, 2, 3, 4, 5, 6 の整数を順番に代入して計算し、最もコストが小さくなる x とそのときの C を出力せよ。
 - (2) 2 次関数の頂点の公式 $x = -b/(2a)$ を用いて x の値を計算し、そのときの最小コスト C を出力せよ。(1) の結果と比較して、どのような違いがあるか確認すること。
-
-

問 4. 【運転免許に関わるお話：制動距離】

車の速度 v (km/h) とブレーキをかけてから止まるまでの距離 d (m) は、近似的に 2 次関数 $d = 0.05v^2 + 0.2v$ でモデル化できる。 v の範囲を 0 から 100 としてグラフを描画せよ。また、速度が $v = 60$ のときの制動距離を Python の変数代入を用いて計算せよ。

本ページは各問の実装方針と解答の要点を示す。

問 1. 直線の当てはめ

```
import matplotlib.pyplot as plt
import numpy as np

# 散布図
x_points = [12, 18, 22, 28, 32]
y_points = [80, 110, 130, 160, 180]
plt.scatter(x_points, y_points, color="red")

# 予測モデル（傾きと切片を調整してフィットさせる）
x_line = np.linspace(10, 35, 100)
# 例:  $y = 5 * x_{\text{line}} + 20$  など、自分で数字を変えてみる
y_pred = 4.8 * x_line + 25
plt.plot(x_line, y_pred, color="blue")
plt.grid(True)
plt.show()
```

問 2. 損益分岐点の計算

```
from sympy import symbols, solve
x = symbols('x')
cost = 200 * x + 15000
revenue = 500 * x

ans = solve(revenue - cost, x)
print("損益分岐点となる個数:", ans) # [50] が出力される
```

問 3. コンピュータによる探索と手計算の比較

```
# (1) 整数による探索
for x in range(1, 7):
    C = 2 * x**2 - 14 * x + 30
    print(f"整数 x={x} のとき C={C}")
# 出力から、x=3 のとき C=6、x=4 のとき C=6 となり、
# 最小値の候補は C=6 となる。

# (2) 公式による計算
a = 2
b = -14
x_opt = -b / (2 * a)
C_min = 2 * x_opt**2 - 14 * x_opt + 30
print(f"解析的な最適解 x={x_opt} のとき C={C_min}")
# 出力結果: x=3.5 のとき C=5.5
# 整数を順番に代入する手法では、小数の谷間 (x=3.5) にある
# 真の最小値 5.5 を見逃す可能性があることがわかる。
```


問 4. 制動距離

```
# グラフ描画
v_vals = np.linspace(0, 100, 100)
d_vals = 0.05 * v_vals**2 + 0.2 * v_vals
plt.plot(v_vals, d_vals)
plt.grid(True)
plt.show()

# 代入による計算
v = 60
d = 0.05 * v**2 + 0.2 * v
print("v=60のときの制動距離:", d, "m") # 192.0 m が出力される
```